

SOLID Race Lab

A Practical, Visual Workshop on the SOLID Principles
Ingeniería de Software II · Universidad Nacional de Colombia · 2026-2

This workshop replaces the discussion-only version with a hands-on one: students complete real, runnable code so that each SOLID principle can be **seen**, not just discussed. The domain is a small console-animated vehicle race — cars, trucks, motorcycles, bicycles and drones racing across the terminal. Every exercise, once solved correctly, can be run to watch the race play out, and is checked automatically with an included pytest suite.

Learning objective

By completing this workshop, students will be able to recognize a violation of each SOLID principle in existing code, explain why it is a problem, and refactor or extend the code to fix it — while keeping a running program working throughout.

Setup (5 min)

- Clone the **solid-race-lab** repository the instructor shared (branch **main** already contains the exercises below).
- Create a virtual environment and install dependencies:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Read **README.md** once before starting — it explains how to run each exercise's visual demo and its automated tests.

How each exercise works (repeat x5)

- Open the exercise file in **exercises/** and read its docstring: it states the problem and a numbered list of TODOs.
- Complete the TODOs directly in that file. Do not modify **engine/track.py**.
- Run the exercise to see the race animate in your terminal:

```
python -m exercises.exN_xxx
```

- Run that exercise's tests to confirm it is solved:

```
pytest tests/test_exN_xxx.py -v
```

An exercise is done when its tests pass **and** the demo runs without errors or unexpected behavior (no crashes, no vehicle moving backwards, etc.).

Exercise 1 — SRP: Single Responsibility Principle

File: `exercises/ex1_srp.py`

Problem. The **Car** class currently does three unrelated jobs: it tracks its own race state, prints its own status line to the console, and writes race results to a log file — all inside **move()**.

Your task:

- Strip **Car** down to only tracking state and moving itself.
- Write a new **RaceLogger** class responsible only for recording and saving race results (**record(tick, racers)**, **save(path)**, **entries**).
- Wire **RaceLogger.record** as the **on_tick** callback passed to **Track.run(...)**, and call **logger.save()** after the race.

Discuss with your group: Why is it better for Car to know nothing about logging or printing? What would break first if a professor asked you to log results as JSON instead?

Exercise 2 — OCP: Open/Closed Principle

File: `exercises/ex2_ocp.py`

Problem. **Vehicle**, **Car** and **Truck** are complete and must not change. **Motorcycle** and **Bicycle** are unfinished.

Your task:

- Implement **Motorcycle.move()** so its step size varies noticeably from tick to tick (fast but erratic).
- Implement **Bicycle.move()** so its step size shrinks over time as the rider tires out, but never drops below 1.
- Do this purely through new code in the subclasses — no edits to **Vehicle**, **Car**, **Truck** or **Track**.

Discuss with your group: If you found yourself wanting to edit **Vehicle** or **Track** to finish this, what does that tell you about your design?

Exercise 3 — LSP: Liskov Substitution Principle

File: `exercises/ex3_lsp.py`

Problem. Every **Vehicle** must honor a contract: **move()** never decreases position and never raises. **UnreliableCar** currently breaks both rules.

Your task:

- Redesign **UnreliableCar.move()** so “unreliability” never violates the **Vehicle** contract — for example, by simply not advancing on some ticks instead of rolling back or raising.
- Leave **Vehicle** and **Track** unchanged.

Discuss with your group: What would you have had to add to **Track** (extra try/except, position clamping...) if you had NOT fixed **UnreliableCar**? Why is fixing the subclass better?

Exercise 4 — ISP: Interface Segregation Principle

File: exercises/ex4_isp.py

Problem. One fat interface, **VehicleActions**, forces every vehicle to implement **refuel()**, **fly()** and **pedal_harder()** even when they make no sense for it.

Your task:

- Replace **VehicleActions** with four small interfaces: **Movable**, **Refuelable**, **Flyable**, **Pedalable** (one abstract method each).
- Make **GasCar** implement only **Movable** + **Refuelable**, and **Bicycle** only **Movable** + **Pedalable**. Delete their now-unnecessary “I can't do that” stub methods.
- Add a new **Drone** class implementing only **Movable** + **Flyable**, and add it to the race in **main()**.

Discuss with your group: Which interface(s) would a future Submarine or Airplane vehicle need? Does adding one require touching any existing vehicle class?

Exercise 5 — DIP: Dependency Inversion Principle

File: exercises/ex5_dip.py

Problem. **Race** currently builds its own **SportsCar** and **DeliveryVan** inside **__init__**, so racing a different roster means editing **Race** itself.

Your task:

- Refactor **Race** to receive its racers (and optionally a **Track**) through the constructor, stored as **self.racers**. **Race** must not import or know about any concrete vehicle class.
- Add a **RocketSled** vehicle class.
- In **main()**, build two different rosters and run **Race** with each, proving **Race** itself never changed.

Discuss with your group: How does injecting the roster make **Race** easier to unit-test without a real terminal animation?

Wrap-up

- Run the full test suite once more (`pytest -v`) — all tests across all five exercises should pass.
- In your group, identify one place in your own course project (or a past project) where one of these five principles was being violated. What would you change?
- Keep `exercises/engine/track.py` handy: the same “racer” abstraction you practiced with in Exercise 5 (DIP) is the same pattern that lets you plug new, unrelated game objects into a shared engine without modifying it.

Study material

See **Material-Estudio-SOLID.pdf** for a theory refresher on each principle, with before/after examples drawn from this same project — useful to read before the workshop or to review afterwards.